第 7-2 讲 · 把它做出来并交付

> **模块 7 · 独立造东西** ｜ 建议用时 **40 分钟**（阅读 20 + 动手 20）
> 前置要求：完成 7-1（你手上应该有一份自己的设计方案）

本节目标

读完并做完这一讲，你能：

1. **把上一讲的设计方案变成可执行的开发计划**（分几步、每步产出什么）。
2. 掌握**"从最小可用版本开始"**的具体做法，以及每一轮该加什么。
3. 知道怎么做**自己的项目骨架**（复用记账簿的结构，不重新发明）。
4. 有一份**完整的自查清单**（从代码到发布，逐项确认）。
5. 明白**课程结束了，但学习才刚开始**——知道下一步该学什么。

一、开工前的准备

1.1 你需要三样东西

需要	从哪来
一份设计方案	上一讲你的产出（需求 + 数据模型 + 流程 + 选型）
一个项目骨架	**复制记账簿的结构**（下面详讲）
一个"完成"的定义	你的验收标准（7-1 讲的 3.3 节）

第二样最省时间，而且**你已经有了**——就是记账簿。

1.2 为什么"复制结构"不是抄

你学会的是模式，不是代码。

```
记账簿的结构：
src/main/          主进程（管数据、系统）
src/preload/       传菜口
src/renderer/      界面
src/shared/        共用的类型和常量
scripts/           验证脚本
```

任何 Electron 应用都长这样。 这不是"抄记账簿"，**这是 Electron 应用的标准结构**（VS Code、Obsidian 也是这个形状）。

你复用的是"骨架"，不是"内容"：

复用（骨架）	自己写（内容）
目录结构	你的数据表
IPC 的四层链路（2-2 讲）	你的业务逻辑
验证脚本的套路（5-1 讲）	你的检查项
打包配置（6-1 讲）	你的产品名和图标
界面组件的基础样式	你的界面

这就是那 24 讲真正的价值：你学的是一套可复用的骨架。

二、开发计划：分五轮走

回顾记账簿的真实开发过程（项目 `docs\` 里就有第一轮到第三轮的报告，后面又追加了截图识别、预算、汇总、应用锁等）：

```
第一版：记一笔 + 列表 + 账户分类          ← 最小可用
第二版：日历、周期记账、附件              ← 让它好用
第三版：动效、图标重新设计                ← 让它好看
第四版：截图识别、预算、汇总、搜索        ← 加高级功能
第五版：应用锁、导入导出、备份、发布      ← 能交付给别人
```

每一轮都是"能用的完整产品"，而不是"半成品"。

给你的建议是按同样的节奏走，但每轮的规模可以更小（因为你是第一次独立做）。

2.1 第一轮：最小可用（目标：1 个周末）

做什么：

任务	产出
搭骨架	能跑起来一个空窗口
建数据库表	你的表结构（照着 7-1 的设计）
打通一条链路	**能新增一条记录并写进数据库**（这是最关键的一步）
显示列表	能看到刚记的东西

关键：**第一条链路要走通全套**（界面 → preload → IPC → 校验 → 仓储 → SQL）。**这一条通了，剩下的功能都是复制这套模式。**

验收：

- [x] 双击能打开
- [x] 能新增一条
- [x] 关掉再打开，记录还在

如果你只做到这里就停下来了——没关系，你已经完成了一次完整的开发流程。

2.2 第二轮：让它好用（目标：1–2 周）

加什么	为什么现在加
修改 / 删除（带软删除 + 撤销）	记错了要能改
搜索 / 筛选	数据多了要能找到
空状态提示	第一次用的人需要引导
表单校验	防止记进错误数据

这一轮结束，它就是一个"真的能用"的工具了。

2.3 第三轮：让它好看（可选，但值得做）

记账簿在这一轮做了：动效、图标重设计、深浅色主题。

为什么值得：

> **软件好不好看，决定了你会不会一直用它。**

一个真实观察：功能一样的两款软件，**好看的那个会被用下去**，难看的那个会被卸载。**因为用软件是一种日常体验，不是一次性任务。**

但注意顺序：**先让它能用，再让它好看。** 反过来会让你陷入"调了半天颜色，功能还没做"的陷阱。

2.4 第四轮：加差异化功能

记账簿在这一轮加的是截图识别——这是它**最特别**的功能。

问自己：

> **"我这个软件，有什么是别的软件没有的？"**

如果没有，那就加一个。**差异化决定了别人为什么要用你的，而不是用现成的。**

2.5 第五轮：交付（模块 5、6 的内容）

做什么	对应讲义
加验证脚本	5-1
检查数据安全	5-2
打包	6-1
写官网、发布	6-2

这一轮做完，别人就能下载使用了。

三、每天怎么工作（一个可复制的节奏）

3.1 一次完整的工作循环

每次动手都走这六步（4-1 讲学的，这里给个更具体的版本）：

```
① 想清楚这次要做什么        →  一句话写下来
② 确认基线干净              →  npm run typecheck（0 错误）
③ 备份要改的文件            →  Copy-Item 到临时目录
④ 改 + 立刻看效果           →  dev 模式开着，改完就刷新
⑤ 跑验证                    →  按改动范围选（5-1 讲）
⑥ 收尾                      →  提交/记录/清理临时文件
```

第 ① 步的价值："一句话写下来"会暴露"其实我还没想清楚"。

如果你写不出这一句话，说明这一步不该开工——先去想清楚。

3.2 卡住的时候怎么办

卡住的情况	怎么办
不知道怎么写（不会的 API）	搜官方文档 → 搜报错 → 找一个最小的例子跑通
报错看不懂	4-4 讲的排查顺序：读原文 → 定位 → 判断类型 → 二分还原
功能越做越复杂	**砍需求**（回到 MVP），把"以后再说"的记到待办清单
卡超过 20 分钟	**问人**（或在社区提问）。附上：完整报错 + 你刚改了什么
完全不想做了	**停下来。**个人项目最大的敌人是"把它变成负担"

最后一条要认真对待：

> **个人项目要能停。** 累了就停几天，别硬撑——**硬撑的结果往往是彻底放弃**，而不是做完。

3.3 一个实用建议：写"开发日志"

每次工作完，用三行记一下：

```
今天做了什么：加了"按金额排序"
遇到什么坑：忘了把 sort 加进 useMemo 依赖数组，点了没反应
下次做什么：加"按分类筛选"
```

为什么有用：

- **下次打开项目时，5 秒就能找回状态**（不用重新读代码）
- **"遇到什么坑"这部分，以后会成为你的经验库**
- 做完一个项目回头看，**这份日志就是你的成绩单**

我在做记账簿时就是这么记的——项目 `docs\` 里那些"第一轮-技术方案""第二轮-报告"，就是这种日志的正式版本。

四、最终自查清单

在你说"做完了"之前，逐项过一遍。 这份清单覆盖了你 24 讲学的全部要点。

4.1 代码质量

#	检查	怎么验
1	类型检查 0 错误	`npm run typecheck`
2	没有 `console.log` 遗留在关键路径	搜一下 `console.log`
3	没有硬编码的绝对路径	搜 `C:\` 或 `/Users/`
4	没有注释掉的死代码	搜 `// TODO` 和整段注释
5	**金额一律用整数分**	搜 `amount`，确认没有浮点运算
6	**SQL 全部用 `?` 占位符**	搜 `INSERT INTO`、`SELECT`，确认没有字符串拼接

4.2 数据安全

#	检查	对应讲义
7	删除是软删除（不是真删）	2-3 / 5-2
8	有回收站或撤销机制	5-2
9	**有备份功能**	5-2
10	备份前调用了 `checkpoint()`（如果用 SQLite）	5-2
11	数据库结构变更走迁移（不改旧迁移）	2-3
12	危险操作有二次确认	5-2

4.3 界面与体验

#	检查	为什么
13	有**空状态**引导（"还没有数据，点这里开始"）	第一次用的人需要知道该干什么
14	出错时给**用户能看懂**的提示（不是技术报错）	3-2 讲的 `toastError` 就是干这个
15	高频操作在**容易点到**的位置	5.2 节的"以任务为中心"
16	删除/危险操作有确认	防误操作
17	长列表能滚动、不卡	性能
18	窗口能调整大小、内容不错位	基本要求

4.4 验证

#	检查	怎么验
19	有自动化检查（哪怕只有几条）	5-1 讲的 `check()` 套路
20	关键业务规则有断言（如"某个数字口径"）	5-1 讲
21	验证脚本用**沙箱**，不碰真实数据	5-1 讲的 `.tmp\` 机制
22	改完代码会跑验证（形成习惯）	4-1 讲的六步流程

4.5 打包与发布

#	检查	怎么验
23	能打出安装包	6-1
24	**打包产物真的能跑**（不只是源码能跑）	6-1 讲的 `verify:packaged`
25	安装包体积合理（没把不该带的打进去）	6-1 的踩坑记录
26	版本号、产品名正确	6-1
27	有"SmartScreen 提示"说明（如果没签名）	6-2
28	有隐私说明（哪怕一句话）	5-3

4.6 最后三条（最重要的）

#	检查	为什么最重要
29	**你自己真的在用它**	**不用它，你不会发现真正的问题**
30	**数据备份到了这台电脑之外**	5-2 讲的终极边界
31	**能对别人讲清它是什么**	5-3 讲的"三句话介绍"

第 29 条特别重要：

> **一个自己都不用的软件，不可能做好。**
> 因为**只有真实使用**才会暴露那些"测试想不到"的问题：用着不方便的地方、数据多了会卡、某个操作要重复点五次……

记账簿的每一轮改进，都来自"我自己用着不舒服"。

五、课程结束了，然后呢

5.1 你现在在哪

回顾你走过的路：

阶段	你做到的
起点	有一个能跑的软件，但不知道它怎么工作的
模块 0–1	能在终端里操作、理解了程序是什么
模块 2	理解了整个软件的骨架（四个部件）
模块 3	**能读懂任何一段代码**
模块 4	**能安全地改代码**（从需求到验收全流程）
模块 5	知道怎么保证质量和安全
模块 6	**能把软件交给陌生人**
模块 7	**能从零设计并做出自己的软件**

这就是"零基础 → 能独立开发"的完整路径。

5.2 还没学的东西（诚实清单）

这套讲义没教你（也不该教，因为用不上）：

没学的	什么时候需要
git 版本控制	**建议尽快学**（半天，收益极大）
测试框架（Jest / Vitest）	项目变大、多人协作时
前端框架深入（React 高级特性）	界面变复杂时
后端 / 服务器	需要多设备同步时
云服务 / 数据库集群	用户量很大时
性能优化	遇到性能问题时
移动端开发	要做手机 App 时

注意第一项：**git 是这套讲义里唯一"我强烈建议你尽快补上"的东西。**

为什么：它有 **半天**的学习成本，但能省掉你**无数次的"改坏了怎么还原"**（4-1 讲里那些手工备份的操作，有了 git 全都不需要）。

5.3 下一步怎么走（三个方向）

方向	适合	做什么
打磨现在的项目	你想让它真的有人用	加功能、修 bug、写教程、推广
做第二个软件	想巩固能力	换个领域（笔记/健身/收藏），**这次会快很多**
补基础	想走得更远	git → 测试 → 算法与数据结构 → 网络

我的建议：**先做第二个软件。**

为什么：因为**第一个项目你是"跟着讲义做的"，第二个才是"你自己做的"。**做完第二个，你才会真正相信"我能独立开发"。

而且第二个会快得多——骨架现成、流程熟悉、坑都踩过了。

5.4 最后一段话

这套讲义教了 24 讲，但真正重要的只有一句话：

> **遇到不懂的东西，你知道怎么查、怎么试、怎么验证——而不是等别人告诉你。**

这就是"能独立开发"的全部含义。

具体说：

遇到	你会
一个陌生的项目	**从入口追一条线**（3-1）
一个报错	**读原文、定位、判断类型、二分还原**（4-4）
一个新需求	**问三个问题、判断类型、列改动清单**（4-2）
一个改不动的 bug	**确认基线、还原改动、逐步缩小**（4-1）
不知道该不该信自己的改动	**跑验证**（5-1）
要交给别人	**打包、验证产物、说明风险**（6-1、6-2）

这六条，就是你这 24 讲真正带走的东西。

六、动手任务（最后一课）

步骤 1：过一遍自查清单

拿出第四节的 31 条清单，对着你自己的项目（如果还没开始做，就对着记账簿）逐项过。

把不通过的记下来——那就是你接下来要做的事。

步骤 2：写你的"开发日志"第一行

新建一个文本文件，写下：

```
项目名：_____
第一版目标：_____（只写 1–2 条核心功能）
预计完成时间：_____
最可能遇到的困难：_____
```

就这五行。 然后**今天就开始做第一步**（搭骨架，让一个空窗口跑起来）。

步骤 3：做一个"最小闭环"

今天（或这个周末）的目标只有一个：

> **让一条数据从界面走到数据库，再从数据库回到界面。**

具体说：

① 界面有一个输入框和按钮
② 点击按钮 → 数据存进数据库
③ 列表显示出来
④ 关掉再打开，数据还在

不要做别的——不要美化、不要加功能、不要想架构。

这一条闭环通了，你就成功了一大半，因为**剩下的功能都是这件事的重复。**

步骤 4：写一段"结课总结"

给自己写三句话：

① 这 24 讲里，我真正学会的是：_____
② 我还完全不懂的是：_____
③ 我接下来最想做的软件是：_____

第 ② 条特别重要——知道自己的边界，才知道下一步学什么。

不要写"我都懂了"——那是自欺。**诚实地写"我不懂 X"，然后把它当成下一个目标。**

卡住了怎么办

现象	原因	怎么办
不知道从哪开始	想太多	**只做步骤 3 的那一条闭环**，别的都别想
骨架搭不起来	正常	**复制记账簿的 `package.json` 和配置**，改名字就行
觉得什么都不会	学完就忘是正常的	**你没忘，只是需要"用一次"才会激活。**做第二个项目时就回来了
项目做不动了	很常见	**停下来，砍需求，回到 MVP**
想继续深入	好	见 5.3 的三个方向，**建议先学 git**

七、自测题

1. 第一轮开发只做"一条闭环"，为什么不要多做?

2. "复制记账簿的结构"和"抄代码"有什么区别?

3. 自查清单里，哪一条是"最重要"的？为什么?

4. 为什么讲义建议"先做第二个软件"，而不是继续打磨第一个?

5. 用一句话概括这 24 讲真正教给你的东西。

答案与解析

1. 因为**第一条闭环（界面 → IPC → 数据库 → 回到界面）打通了，剩下的功能都是它的重复。** → 而且做闭环的过程中你会遇到大量"没想到的细节"（校验、刷新、错误处理），**这些细节才是真正花时间的。**先做别的会累积风险。

2. 复用的是**骨架**（目录结构、四层链路、验证套路、打包配置），这些是**任何 Electron 应用的标准形状**（VS Code、Obsidian 也一样）；**自己写的是内容**（数据表、业务逻辑、界面）。 → **学的是模式，不是代码。**

3. 第 29 条：你自己真的在用它。 → 因为**只有真实使用才会暴露那些"测试想不到"的问题**（用着不顺手、数据多了会卡、某个操作要重复点五次）。**记账簿每一轮改进都来自"我自己用着不舒服"。**

4. 因为**第一个项目你是"跟着讲义做的"，第二个才是"你自己做的"。**做完第二个，你才会真正相信"我能独立开发"。 → 而且第二个会**快得多**（骨架现成、流程熟悉、坑都踩过）。

5. 遇到不懂的东西，你知道怎么查、怎么试、怎么验证——而不是等别人告诉你。

八、术语表（最后一讲，只收最关键的）

术语	一句话解释
MVP / 最小可用版本	只保留核心功能、能立刻用起来的第一版
最小闭环	数据从界面到数据库再回来的完整一条路
项目骨架	可复用的目录结构与配置，不含业务内容
开发日志	每次工作后记三行，帮助下次快速找回状态
自查清单	交付前逐项确认的检查表
版本控制 / git	记录代码历史、随时回退（**建议尽快学**）
技术债	为了当下能跑通而做的妥协，将来要还的账

九、模块 7 与全课程小结

讲	你学到的
7-1	**四步设计法**：需求 → 数据 → 界面 → 选型；MVP 的意义
7-2	**开发计划、工作节奏、自查清单**；下一步该学什么

全课程回顾：

模块	你获得的能力
0–1	能在终端操作、理解程序、文件、构建
2	理解任意 Electron 应用的四大部件
3	**能读懂陌生代码**
4	**能安全地改代码**（全流程）
5	能保证质量与数据安全
6	能把软件交付给陌生人
7	**能从零设计并做出自己的软件**

十、最后

你现在手里有的东西：

1. **一个完整的、真实的、能用的软件**（记账簿），作为你学习过程的"标本"
2. **一套可复用的开发骨架**（目录结构、四层链路、验证套路、打包配置）
3. **一套方法论**（读代码、改代码、验证、发布）
4. **一个判断力**：知道什么该做、什么不该做、什么时候该停

这四样东西合起来，就是"能独立开发"。

接下来只差一件事：**去做你自己的那个软件。**

去做吧。做完记得回来告诉我 —— 或者，**当你卡住的时候也欢迎回来**，那时候你手里应该有一份完整的报错，和一句"我改了这些"。

祝你做出好东西。

讲义版本：v1 ｜ 对应项目版本：0.1.0 ｜ 全课程 24 讲 · 完